I've left the implementation of `MinimumAgeHandler` for `MinimumAgeRequirement` as an exercise for the reader, as it's similar to the handlers you have already seen. You can find an example implementation in the code samples for the chapter.

TIP It's possible to write generic handlers that can be used with multiple requirements, but I suggest sticking to handling a single requirement. If you need to extract some common functionality, move it to an external service, and call that from both handlers.

This pattern of authorization handler is common, but in some cases, instead of checking for a *success* condition, you might want to check for a failure condition. In the airport example, you don't want to authorize someone who was previously banned from the lounge, even if they would otherwise be allowed to enter.

You can handle this scenario by using the `context.Fail()` method exposed on the context, as shown in the following listing. Calling `Fail()` in a handler always causes the requirement, and hence the whole policy, to fail. You should use it only when you want to guarantee failure, even if other handlers indicate success.

Listing 24.10 Calling `context.Fail()` in a handler to fail the requirement

```
public class BannedFromLoungeHandler :
    AuthorizationHandler<AllowedInLoungeRequirement>                    ❶
{
    protected override Task HandleRequirementAsync(                     ❷
        AuthorizationHandlerContext context,                            ❷
        AllowedInLoungeRequirement requirement)                         ❷
    {
        if(context.User.HasClaim(c => c.Type == "IsBannedFromLounge"))  ❸
        {
            context.Fail();                                             ❹
```